

Шпаргалка по Go

Синтаксис, срезы и карты, ошибки, структуры и интерфейсы, горутины и каналы, модули и инструменты командной строки. Примеры для Go 1.21 и новее.

Первая программа

```
package main

import "fmt"

func main() {
    fmt.Println("Привет")
}
```

`go run .`
Собрать и сразу выполнить пакет в текущем каталоге

`go build ./...`
Собрать все пакеты проекта

`go mod init example.com/app`
Создать модуль: появляется `go.mod`

Переменные и типы

`var x int = 10`
Полная форма объявления

`y := 10`
Короткая форма с выводом типа, только внутри функций

`const Limit = 100`
Константа, вычисляется на этапе компиляции

`int, int64, float64, bool, string, byte, rune`
Базовые типы; `rune` — символ Unicode, `byte` — то же, что `uint8`

`var p *int = &x; fmt.Println(*p)`
Указатель: взятие адреса и разыменование

`type UserID int`
Свой тип на основе существующего

Нулевые значения: `0` для чисел, `""` для строк, `false` для `bool`, `nil` для указателей, срезов, карт и интерфейсов.

Срезы и массивы

`nums := []int{1, 2, 3}`
Срез — основная последовательность в Go

`make([]int, 0, 10)`
Срез нулевой длины с запасом ёмкости

`nums = append(nums, 4)`
Добавление; `append` возвращает новый срез, результат надо присвоить

`nums[1:3]`
Подсрез; правая граница не включается, память общая с исходным

`len(nums), cap(nums)`
Длина и ёмкость

`copy(dst, src)`
Скопировать элементы, чтобы разорвать связь по памяти

`var arr [5]int`
Массив фиксированной длины; длина — часть типа

Карты

`ages := map[string]int{"аня": 30}`
Карта: ключ — значение

`ages["петя"] = 25`
Запись; чтение отсутствующего ключа даёт нулевое значение

`v, ok := ages["вася"]`
Проверка наличия ключа

`delete(ages, "аня")`
Удаление

Осторожно: запись в `nil`-карту вызывает панику. Создавайте её через `make(map[string]int)` или литералом.

Управление потоком

```
if err := doWork(); err != nil {
    return err
}

for i := 0; i < 10; i++ {
    for i, v := range nums {
        for { break } // бесконечный
    }
}

switch {
case x > 10:
    ...
default:
    ...
}
```

В Go нет `while`: его роль играет `for` с одним условием. В `switch` ветки не проваливаются — `break` не нужен.

Функции и ошибки

```
func divide(a, b float64) (float64, error) {
    if b == 0 {
        return 0, errors.New("деление на ноль")
    }
    return a / b, nil
}

res, err := divide(10, 0)
if err != nil {
    return fmt.Errorf("расчёт: %w", err)
}
defer file.Close()
```

`%w` в `fmt.Errorf`
Оборачивает ошибку с сохранением исходной

`errors.Is(err, os.ErrNotExist)`
Проверка конкретной ошибки в цепочке

`errors.As(err, &target)`
Извлечь ошибку нужного типа

`defer`
Отложить вызов до выхода из функции; выполняется и при панике

Структуры и методы

```
type User struct {
    Name string `json:"name"`
    Email string `json:"email,omitempty"`
}

func (u User) Greeting() string {
    return "Привет, " + u.Name
}

func (u *User) Rename(name string) {
    u.Name = name
}
```

Метод с указателем нужен, чтобы менять поля. Имя с заглавной буквы делает поле или функцию доступными из других пакетов.

Интерфейсы

```
type Storer interface {
    Get(id string) ([]byte, error)
    Put(id string, data []byte) error
}
```

Тип реализует интерфейс автоматически, если у него есть нужные методы — объявлять это не требуется. Небольшие интерфейсы удобнее: часто достаточно одного метода.

```
v, ok := x.(MyType)
    Приведение типа с проверкой

switch v := x.(type) { case int: ... }
    Ветвление по фактическому типу
```

Горутины и каналы

```
go worker(ch)                // запуск горутины

ch := make(chan int, 10)      // канал с буфером
ch <- 1                        // отправка
v := <-ch                     // приём
close(ch)

var wg sync.WaitGroup
wg.Add(1)
go func() {
    defer wg.Done()
    ...
}()
wg.Wait()

select {
case v := <-ch:
    ...
case <-ctx.Done():
    return ctx.Err()
}
```

```
sync.Mutex
    Защита общих данных: Lock и Unlock

context.WithTimeout
    Ограничение времени и отмена операции

go run -race ./...
    Поиск гонок при обращении к общим данным
```

Обобщения

```
func Map[T, R any](in []T, f func(T) R) []R {
    out := make([]R, 0, len(in))
    for _, v := range in {
        out = append(out, f(v))
    }
    return out
}
```

Инструменты

```
go test ./... -v
    Запуск тестов во всех пакетах

go test -run TestName -cover
    Один тест; с покрытием

go test -bench . -benchmem
    Замеры производительности и выделений памяти

go fmt ./... / gofmt -w .
    Единое форматирование — обязательная привычка в Go

go vet ./...
    Поиск подозрительных мест, которые компилятор пропускает

go mod tidy
    Добавить недостающие и убрать лишние зависимости

go get example.com/pkg@v1.2.3
    Добавить или обновить зависимость

GOOS=linux GOARCH=amd64 go build
    Сборка под другую платформу

go doc fmt.Println
    Документация по пакету или функции в терминале
```

Тест

```
func TestDivide(t *testing.T) {
    got, err := divide(10, 2)
    if err != nil {
        t.Fatalf("неожиданная ошибка: %v", err)
    }
    if got != 5 {
        t.Errorf("получили %v, ожидали 5", got)
    }
}
```

Файл теста называется *_test.go и лежит рядом с кодом. Таблица случаев плюс t.Run — обычный способ описать много вариантов.